

# Microservices, Kubernetes & DevOps

Skalierbare digitale Prozessarchitektur bauen –  
von Servicegrenzen zu einem belastbaren Operating Model.

*Lehrskript – Tag 10 von 16*

Lernpfad:  
Wissen → Anwendung → Management

Bearbeitungsdauer:  
1 Kurstag

Dozent:  
Can Siebert

Format:  
Theorie · Fallstudie · Coaching

---

## Lernziele dieses Tages

Am Ende dieses Tages verstehen Sie, warum eine skalierbare digitale Prozessarchitektur nicht aus Werkzeugen, sondern aus *Entscheidungen* besteht. Sie können Servicegrenzen entlang fachlicher Capabilities ziehen, ein Kubernetes-Betriebskonzept skizzieren, eine Teststrategie nach TDD-Prinzipien aufstellen und ein Operating Model entwerfen, das Ownership und Governance leichtgewichtig, aber verbindlich verankert. Drei Stufen – Wissen, Anwendung, Management – bauen aufeinander auf.

### L1 – Wissen (Reproduktion und Einordnung)

- Microservices-Grundlogik kennen: unabhängige Deployments, Servicegrenzen entlang Capabilities, API/Event-Integration, Idempotenz.
- Container und Kubernetes konzeptionell einordnen: Deploy, Skalierung, Self-Healing, Rollback, Service-Discovery, Konfiguration/Secrets.
- TDD-Zyklus (Red-Green-Refactor) und Testpyramide (Unit, Integration, End-to-End) als Qualitätsarchitektur verstehen.
- DevOps-Praktiken und Operating-Model-Bausteine (Produktteams, Plattformteam, Security, Architektur-Governance) einordnen.

### L2 – Anwendung (Transfer auf konkrete Situationen)

- Einen Monolithen entlang Capabilities in fünf bis acht Service-Kandidaten zerlegen und Steckbriefe formulieren.
- Einen Release-Flow mit zwei Quality Gates definieren und eine Testpyramide pro Service dimensionieren.
- Ein Kubernetes-Betriebskonzept (Rollout/Rollback, Autoscaling, Monitoring/Alerting) auf einen Fallprozess anwenden.
- Ein Operating Model entwerfen, das Produktteams, Plattformteam und Security in einer RACI darstellt.

### L3 – Management (Entscheidung und Verantwortung)

- Top-10-Entscheidungen für eine skalierbare Plattform priorisieren (Servicegrenzen, Datenhoheit, SLOs, Standards).
- Zielkonflikte Speed vs. Safety, Autonomie vs. Standardisierung, Innovation vs. Audit transparent entscheiden.
- Reliability- und Risiko-Architektur (SLOs, Error Budgets, Change Failure Rate, MTTR) als Steuerungsinstrument nutzen.
- Entscheidungen unter Unsicherheit mit Frühwarnsignalen absichern statt sie zu vermeiden.

#### Roter Faden

Microservices sind kein *Technologiethema*, sondern eine *Organisationsentscheidung*. Kubernetes löst die Skalierung – aber nur, wenn die Organisation Skalierung verantworten kann. Wer die Plattform baut, ohne das Operating Model zu klären, baut ein leistungsfähigeres Chaos. (vgl. Newman, 2021, Kap. 1; Skelton & Pais, 2019)

## 1. Einstieg: Warum klassische Architektur an Grenzen stößt

Die meisten gewachsenen Unternehmen kennen die gleiche Situation: ein monolithisches Kernsystem, das vor zehn bis fünfzehn Jahren begonnen wurde, mittlerweile alle Geschäftsprozesse trägt, und in jedem Release alle Bereiche gleichzeitig betrifft. Die Release-Zyklen sind lang (sechs bis acht Wochen sind keine Seltenheit), die Tests aufwendig, die Schmerzgrenze der Beteiligten hoch. Jedes neue Feature trägt das Risiko, etwas anderes zu beschädigen – weshalb Vorsicht zur Tugend wird und Innovation zur Ausnahme. (vgl. Newman, 2021, Kap. 1)

In dieser Situation klingt die Antwort einfach: *Microservices*. Wir zerlegen den Monolithen in kleine, eigenständige Dienste, deployen jeden für sich, skalieren punktgenau und befreien die Teams. Die Realität ist nüchterner. Ein schlecht geschnittener Microservice ist kein Fortschritt, sondern eine teurere Form desselben Problems – nur in mehreren Repositories und mit zusätzlichen Schnittstellenkosten.

### 1.1 Die drei Schmerzpunkte des Monolithen

1. **Kopplung im Code.** Eine Änderung in einem Modul zwingt zur Vollregression. Tests sind teuer, Deployments selten, Eingriffsrisiko hoch.
2. **Kopplung im Datenmodell.** Ein zentrales Schema, das alle Bereiche teilen. Schemaänderungen erfordern Verhandlungen mit allen Anrainern.
3. **Kopplung in der Organisation.** Ein Release bedeutet ein Synchronisationsfenster für alle Teams – mit Steuerungsaufwand, der mit der Größe quadratisch wächst.

### 1.2 Warum Microservices keine Silberkugel sind

Microservices versprechen Entkopplung – liefern sie aber nur dann, wenn die Servicegrenzen *entlang fachlicher Capabilities* verlaufen, nicht entlang technischer Schichten oder Datenbanktabellen. Ein typischer Anti-Schnitt: “Datenbank-Service”, “UI-Service”, “Logik-Service”. Drei Services, dieselbe Kopplung – nur jetzt über Netzwerkgrenzen, mit Latenz und neuen Fehlerklassen. (Newman, 2021, Kap. 3; Evans, 2003)

Ein guter Servicegrenzen-Schnitt hingegen folgt der Frage: *Was ist eine eigenständige Geschäftsfähigkeit?* “Order”, “Inventory”, “Shipping”, “Billing” sind Capabilities – sie haben eigene Daten, eigene Entscheidungen, eigene Owner. Wenn eine Capability sich ändert, ändert sich *ein* Service, nicht das halbe Unternehmen.

### 1.3 Was “skalierbar” wirklich bedeutet

Skalierbarkeit hat drei Dimensionen, die oft verwechselt werden:

- **Last-Skalierung.** Der Service verträgt zehntausend statt hundert Requests pro Sekunde. Eher ein Plattformthema (Kubernetes, Autoscaling).
- **Team-Skalierung.** Mehr Teams können parallel an unterschiedlichen Services arbeiten, ohne sich gegenseitig zu blockieren. Eher ein Architekturthema (Servicegrenzen).
- **Entscheidungs-Skalierung.** Die Organisation kann hundert Entscheidungen pro Woche treffen, ohne im Architekturboard zu ersticken. Eher ein Governance-Thema (Leitplanken, Autonomie).

Eine Plattform ist erst dann skalierbar, wenn alle drei Dimensionen funktionieren. Wer nur Last skaliert, baut leistungsfähige Engpässe. Wer nur Teams skaliert, produziert Architekturchaos. Wer nur Entscheidungen skaliert, riskiert Inkonsistenz.